Redux

Redux is a predictable state container for JavaScript apps, mainly used with React but also usable elsewhere. It helps manage global state in a structured and predictable way.

1. **Store – a single source of truth.**
   Add all slices as key value pair

2. **Actions – plain objects that describe what happened.**

   ```
   { type: "INCREMENT" }
   ```

   The event triggered

3. **Reducers – pure functions that define how state changes based on actions.**

   ```
   const counterReducer = (state = 0, action) => {
    switch(action.type) {
      case "INCREMENT": return state + 1;
      case "DECREMENT": return state - 1;
      default: return state;
    }
   }
   ```

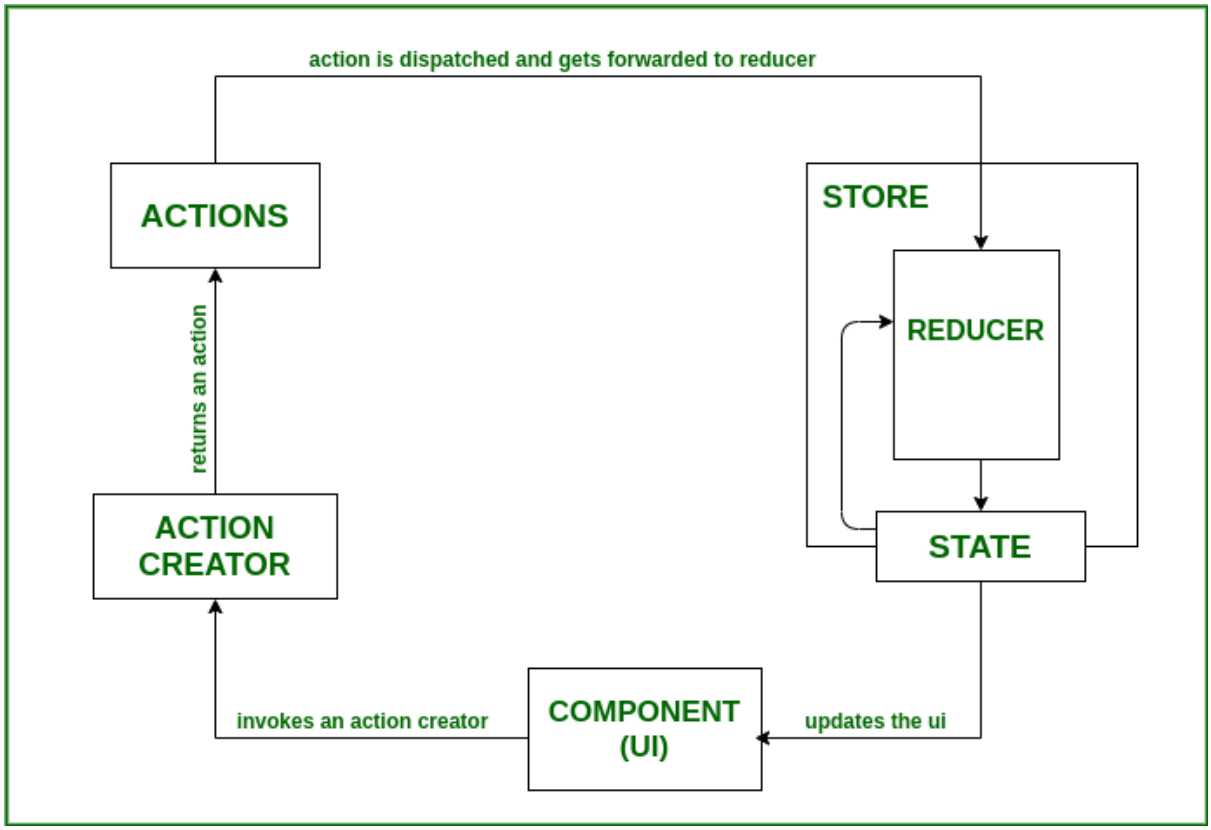
   function called by action

4. **Dispatch – a function to send actions to the store.**

   ```
   store.dispatch({ type: "INCREMENT" });
   ```

   event handling with dispatch triggering action

5. **Selectors – functions to select a slice of state (optional but recommended).**



2. Why Redux?

React itself has local state (useState) and Context API. So why Redux?

2. Why Redux?

React itself has local state (useState) and Context API. So why Redux?

How Context API Rendering Works

- **Context API triggers a re-render in all consuming components whenever the value of the provider changes, even if a component only needs part of the value.**
- **Example:**

```
<CounterContext.Provider value={{ count, increment, decrement }}>
  <ComponentA />   // uses count
  <ComponentB />   // only displays a label, doesn't use count
</CounterContext.Provider>
```

- **Problem: When count changes, both ComponentA and ComponentB re-render, even if ComponentB doesn't use count.**
- **This happens because React cannot know which part of the context value a component actually depends on — any change to the object triggers a re-render for all consumers.**

Why This Can Hurt Performance

- **In a small app, it's fine.**
- **In a large app with many components, frequent updates in context cause unnecessary re-renders, which can slow down the UI.**
- **Example: Imagine count updates 60 times per second — every consuming component will re-render.**

2. How Redux Solves This

- **Both Context API and Redux provide global state storage.**
- **Context API: every component that consumes the context re-renders whenever the context value changes, even if the part of the state the component cares about hasn't changed.**
- **Redux: each component subscribes only to the slice of state it actually uses (via useSelector or connect).**

  `const count = useSelector(state => state.counter.count);`

  - **If another slice updates, your component won't re-render.**
  - **Only components using the updated slice re-render.**

✅ **Result: Redux gives fine-grained control over re-renders, which improves performance in larger, frequently-updating apps.**

Problems Redux solves:

1. **Predictability**
    - **State changes are centralized and happen in a predictable manner.**
    - **Every change is traceable via actions, making debugging easier.**
2. **Single Source of Truth**
    - **One store for the whole app → easier to understand and debug.**
3. **Ease of Testing**
    - **Reducers are pure functions → easy to test independently.**
4. **Debugging Tools**
    - **Redux DevTools let you see every action and state change.**
    - **You can time travel: go back and forward between states.**
5. **Middleware Support**
    - **Intercept actions for logging, async calls, error handling, analytics, etc.**
    - **Examples: redux-thunk, redux-saga.**
6. **Decoupling**
    - **Components don't directly manipulate global state; they dispatch actions.**

Redux in React

Step 0: Setup

1. **Make a new React app:**

   npx create-react-app redux-counter
   cd redux-counter

2. **Install Redux and React-Redux:**

   npm install @reduxjs/toolkit react-redux

@reduxjs/toolkit → modern way to write Redux without boilerplate.

react-redux → connects React to Redux store.

Step 1: Create a Redux Slice

What is a slice?
A slice is a single piece of state and all its logic (reducers + actions) combined.
Redux Toolkit makes it super simple with createSlice.

1. **Create folder: src/redux/**
2. **Create file: counterSlice.js**

```
// src/redux/counterSlice.js
import { createSlice } from "@reduxjs/toolkit";

// Step 1: Define initial state
const initialState = {
  count: 0
};

// Step 2: Create slice
export const counterSlice = createSlice({
  name: "counter",       // slice name
  initialState,          // starting state
  reducers: {            // functions to handle actions
    increment: state => {
```

```
    state.count += 1;   // Immer library allows "mutating" state safely
  },
  decrement: state => {
    state.count -= 1;
  },
  reset: state => {
    state.count = 0;
  }
 }
});
```

```
// Step 3: Export actions to use in components
export const { increment, decrement, reset } = counterSlice.actions;

// Step 4: Export reducer to add to store
export default counterSlice.reducer;
```

Explanation of steps:

- **Step 1: initialState → the default state of this slice.**
- **Step 2: createSlice → automatically creates action types and reducer for us.**
- **Step 3: Export actions (increment, decrement, reset) → components will call these to change state.**
- **Step 4: Export reducer → will go into the store.**

Step 2: Configure the Redux Store

What is the store?
The store is the single source of truth for your app's state. All slices are combined here.

Create store.js in src/redux/:

```
// src/redux/store.js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "./counterSlice";

// Step 1: Configure store
export const store = configureStore({
  reducer: {
    counter: counterReducer // adding our slice reducer
  }
});

// Redux DevTools enabled by default
```

Explanation:

- **configureStore → modern way to create store, no deprecated createStore.**
- **reducer → combine slices here. Right now we only have counter.**
- **Redux DevTools and middleware are automatically configured. ✅**

Step 3: Connect Redux to React

Wrap the whole app in a <Provider> so all components can access the store.

Edit src/main.js:

Edit src/main.js:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux"; // step 1
import { store } from "./redux/store";   // step 2

// Step 3: Wrap app with Provider
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Explanation:

- **Provider → makes the Redux store available to every component inside <App>.**
- **Without this, components cannot access the state.**

Step 4: Create Counter Component

We now create the UI and connect it to Redux.

```
// src/Counter.js
import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement, reset } from "./redux/counterSlice";

const Counter = () => {
  // Step 1: Get state from store
  const count = useSelector(state => state.counter.count);

  // Step 2: Get dispatch function
  const dispatch = useDispatch();

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>Counter: {count}</h1>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </div>
  );
};
```

to pass args in dispatch

```
dispatch(increment(3))
```

in slice action

```
increment=(state,action)=>{
        data=action.payload; //3
}
export default Counter;
```

Step-by-step explanation:

1. **useSelector → subscribes to store state and gets the count.**
2. **useDispatch → returns dispatch function to send actions to the store.**
3. **Dispatch actions (increment, decrement, reset) → reducer updates the state → UI automatically re-renders.**